

PROJECT FORESIGHT

TECHNICAL DOCUMENTATION

Weekly Demand Forecasting & Inventory Intelligence System

Client: NorthBay Living

Author: Senior Data Science Team (Zidio Development)

Status: Production Ready & Restored v1.0

Date: July 2026

PROJECT FORESIGHT: FINAL PORTFOLIO & TECHNICAL REPORT

Weekly Demand Forecasting & Inventory Intelligence System for NorthBay Living

1. Executive Summary

Project FORESIGHT is an end-to-end, production-ready machine learning and decision-intelligence system developed for NorthBay Living, a fast-growing home and lifestyle direct-to-consumer brand. By automating data cleaning, feature engineering, weekly demand forecasting, and inventory risk-scoring, Project FORESIGHT replaces intuition-based planning with a quantitative, data-driven approach.

The forecasting module uses a **Random Forest Regressor** trained on weekly aggregated time-series demand. Backtest evaluation shows a **9.86% WAPE**, outperforming the seasonal-naïve baseline model (WAPE of 23.46%) by **13.60%**. This improvement secures potential revenues at risk due to stockouts and frees up working capital locked in overstocked products, demonstrating clear, actionable financial and operational value.

2. Problem Statement

Historically, NorthBay Living managed its supply chain and stock replenishments via manual spreadsheets and merchant intuition. This approach suffered from several limitations:

- **Frequent Stockouts:** Bestsellers frequently stocked out because lead times and demand spikes were not factored in systematically, leading to missed revenue.
- **Excess Capital Lockup:** Slower-moving products were over-ordered during seasonal promotions, locking up working capital in warehouse holding fees.
- **Siloed Data:** Inconsistencies between sales transactions, SKU metadata, and calendars made it difficult to form a unified view of demand.

3. Business Objectives

The system was built around four core objectives:

1. **Demand Projection:** Generate SKU-level demand forecasts over a 6-week forward-looking horizon.
2. **Early Warning Alerts:** Flags high-risk stockout or overstock conditions before they impact operations.
3. **Actionable Recommendations:** Categorize SKUs into operational decision quadrants (**REORDER NOW**, **MARKDOWN** / **CLEAR**, **WATCH** / **VOLATILE**, **HEALTHY**).
4. **Financial Impact Quantification:** Calculate the exact Rupee value of sales at risk and capital locked in excess holdings.

4. Dataset Description

The system integrates four datasets, covering a historical timeline from **January 1, 2024, to January 4, 2026**:

1. **Sales Transactions (sales_daily.csv):** Daily sales records containing transaction dates, SKU IDs, units sold, unit prices, revenues, and promotion flags.
2. **SKU Master (sku_master.csv):** Metadata containing SKU IDs, categories, subcategories, unit costs, list prices, and product launch dates.
3. **Calendar Events (calendar.csv):** Date attributes including week numbers, months, seasons, holiday indicators, and named promotional events.
4. **Inventory Snapshots (inventory_snapshots.csv):** SKU-level snapshots showing units on hand, units on order, manufacturer lead times, and safety thresholds.

5. Data Cleaning & Preprocessing

To establish a single source of truth, **src/data_prep.py** runs a multi-step data cleaning pipeline:

- **Deduplication:** Removes duplicate records.

- **Type Coercion:** Ensures numeric columns are formatted correctly and dates are parsed as standard datetimes.
- **Revenue Consistency:** Validates that daily **revenue** exactly equals **units_sold * unit_price**. Discrepancies are automatically recalculated.
- **Outlier Capping:** Identifies sales spikes per SKU using a 3x Interquartile Range (IQR) threshold. Extreme values are capped, and the associated revenues are recalculated.
- **Referential Integrity:** Discards records referencing invalid or missing SKU IDs.
- **Inventory Bounds:** Clips negative inventory inputs to zero and establishes a default lead time (14 days) and reorder point (10 units) where missing.
- **Audit Logging:** Saves a preprocessing audit summary (**logs/pipe_line_run_log.json**) tracking cleaning metrics.

6. Feature Engineering

The feature engineering module (**src/feature_engineering.py**) operates on a daily grid. For each SKU, it generates 15 features across four categories:

1. **Lags:** Lag 1, Lag 7, Lag 14, Lag 28 (days ago).
2. **Rolling Statistics:** Rolling Mean, Rolling Std, Rolling Max, and Rolling Min (computed over a 7-day window, shifted by 1 day to prevent look-ahead leakage).
3. **Calendar Features:** Day of Week, Week Number, Month, Quarter.
4. **Flags:** Holiday Flag (binary), Promotion Flag (binary), Weekend Flag (1 if Day of Week is Saturday/Sunday, else 0).

These daily features are aggregated weekly. Summing daily lags (7, 14, 28) over the week translates to weekly lags (1, 2, 4 weeks ago), preserving the temporal structure of the demand curves.

7. Forecasting Model

The modeling engine (**src/train_forecast.py**) uses a **Random Forest Regressor** (100 estimators, trained on weekly aggregated demand).

- **Validation:** 5-fold `TimeSeriesSplit` cross-validation on historical training data to evaluate out-of-fold generalization without leakage.
- **Recursive Forecasting:** Future demand forecasts are generated recursively week-by-week. Each step's prediction is updated in the sales lookup database to serve as lagged inputs for subsequent steps.
- **Confidence Intervals:** The standard deviation of the training residuals (σ) serves as the baseline error. For each forward step h , the standard error scales as $\sigma \times \sqrt{h}$ to model uncertainty propagation. The upper and lower bounds represent an 80% confidence interval ($z = 1.28$).
- **Serialization:** Fitted estimators, encoders, and feature column indexes are serialized to `models/forecast_model.pkl`.

8. Evaluation Metrics & Model Comparison

Models are evaluated on a chronological backtest window from **November 20, 2025, to December 31, 2025**, comparing the Random Forest model against a **Seasonal-Naive baseline** (using actual sales from 52 weeks / 364 days ago).

Evaluation Results

- **WAPE Improvement:** The Random Forest model achieves a **9.86% WAPE**, representing a **+13.60% improvement** over the baseline model (WAPE of 23.46%).
- **MAE & RMSE:** Error magnitude is halved, indicating much tighter fits.
- **Forecast Bias:** The ML model shows a minimal positive bias of **+0.33%**, indicating a highly balanced and unbiased demand prediction.

Metric	Seasonal-Naive Baseline	Random Forest ML Model
WAPE	23.46%	9.86%
MAE	3.97	1.67
RMSE	5.80	2.54
Bias	-2.32%	+0.33%

9. Streamlit Dashboard

The dashboard (`app.py`) provides an interactive interface for supply chain operators:

- **Executive Decision Board:** Displays the **Stockout vs. Overstock Risk Matrix** where every SKU is plotted on a grid (bubble size indicates the financial impact at stake). Includes reorder lists and overstock markdown clearance tables.
- **AI Recommendation Panel:** Generates actionable, prioritized reorder quantities and suggested markdown discount strategies (e.g., 30% clearance discount for items exceeding 12 weeks of supply).
- **SKU Deep Dive:** Displays historical sales, baseline, and ML forecasts with shaded 80% confidence bands, alongside an inventory gauge comparing on-hand units to the reorder point (ROP).
- **Data Pipeline Health:** Exposes data cleaning audit metrics and model accuracy KPIs.

10. Business & Financial Impact

Project FORESIGHT drives financial value in several ways:

- **Prevents Revenue Stockouts:** Prioritizing items in the **REORDER NOW** quadrant protects sales by identifying supply deficits.
- **Unlocks Working Capital:** Identifying items in the **MARKDOWN / CLEAR** quadrant helps release capital locked in overstocks through targeted discounts.
- **Logistical Optimization:** Factoring in lead times helps smooth ordering schedules, reducing the need for expensive air freight.

11. Limitations & Future Improvements

Limitations

- **New Product Cold Start:** For SKUs with less than 28 days of history, lag features default to 0.
- **Static Lead Times:** Assumes lead times are constant based on historical inventory records.

Future Improvements

- **Dynamic Lead Time Integration:** Connect with shipping and carrier APIs to adjust lead times in real time based on carrier delays.
- **Merchandiser Promotion Inputs:** Allow operators to input planned promo schedules directly into the dashboard to dynamically shift forecast curves.
- **Sequence Models:** Evaluate sequence models such as Temporal Fusion Transformers (TFT) or LSTMs to improve performance on highly sparse SKU sales.